

Firebase 앱 로그 기반 데이터 분석

다룰 데이터는 음식 앱의 데이터입니다.

Google BigQuery를 사용해 유저 로그 데이터에서 여러 가지 시각화를 추출, 분석해 보았습니다.

스키마	세부정보	미리보기	데이터 탐색기	미리보기	통계	계보	데이터 프로파일	데이터 품질
-----	------	------	---------	------	----	----	----------	--------

≡ 필터

속성 이름 또는 값 입력

☐	필드 이름	유형	모드	키	대조	기본값	정책 태그 ?	설명
☐	event_date	DATE	NULLABLE	-	-	-	-	-
☐	event_timestamp	INTEGER	NULLABLE	-	-	-	-	-
☐	event_name	STRING	NULLABLE	-	-	-	-	-
☐	▶ event_params	RECORD	REPEATED	-	-	-	-	-
☐	user_id	INTEGER	NULLABLE	-	-	-	-	-
☐	user_pseudo_id	STRING	NULLABLE	-	-	-	-	-
☐	platform	STRING	NULLABLE	-	-	-	-	-

스키마 수정

행 액세스 정책 보기

데이터 원본 :

스키마	세부정보	미리보기	데이터 탐색기	미리보기	통계	계보	데이터 프로파일	데이터 품질
-----	------	------	---------	------	----	----	----------	--------

행	event_date	event_timestamp	event_name	event_params.key	event_params.value.string_value	event_int_value	user_id	user_pseudo_id	platform
1	2022-12-16	167113266...	click_cart	firebase_screen	food_detail	null	47029	9583684665.7771756969	iOS
				food_id		1528			
				session_id	ca9d0653-dbf8-4a21-82bd-705...	null			
2	2022-12-16	167114554...	click_cart	firebase_screen	food_detail	null	2264	5135435278.5547245761	iOS
				food_id	null	1526			
				session_id	da439305-b424-4044-8eb5-09...	null			
3	2022-12-16	167115003...	click_cart	firebase_screen	food_detail	null	5946	7190303792.5515746562	iOS
				food_id	null	1704			
				session_id	598ea4de-dc7a-4279-bfa8-a7c...	null			
4	2022-12-16	167115655...	click_cart	firebase_screen	food_detail	null	17961	9905568710.4930728888	iOS
				food_id	null	1509			
				session_id	8d8fb6c1-1592-4018-b4ca-15e...	null			
5	2022-12-16	167115709...	click_cart	firebase_screen	food_detail	null	19552	6782802332.3069304747	iOS
				food_id	null	1289			
				session_id	1636250a-9e89-4a53-93df-d2b...	null			
6	2022-12-16	167115644...	click_cart	firebase_screen	food_detail	null	50538	6878157887.9916936886	iOS
				food_id	null	1091			
				session_id	496a8f39-b447-4cd5-b3fa-c24...	null			
7	2022-12-16	167115990...	click_cart	firebase_screen	food_detail	null	9223	2494553230.1961375915	iOS
				food_id	null	1704			
				session_id	e8274c6e-2dac-4642-8be8-7dc...	null			
8	2022-12-16	167116380...	click_cart	firebase_screen	food_detail	null	48090	9333278191.7729078597	iOS
				food_id	null	1635			
				session_id	8c9d7313-e5f8-48d1-8187-af4...	null			

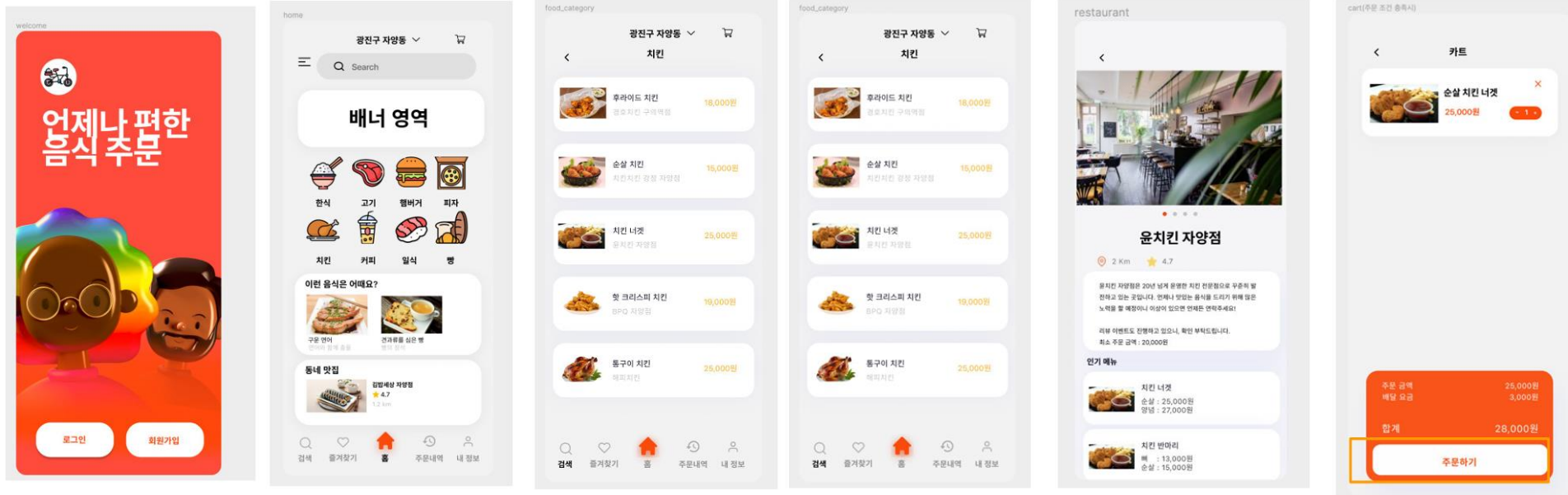
Firestore 앱 로그 기반 데이터 분석

<1. 퍼널 분석>

Welcome -> Home -> Food Category -> Restaurant -> Cart -> 주문하기 클릭 순으로 각 퍼널의 유저 수를 집계

screen_view(welcome) screen_view(home) screen_view(food_category) screen_view(restaurant) screen_view(cart) click_payment(cart)

데이터 : 2022-08-01 ~ 2022-08-18



Firebase 앱 로그 기반 데이터 분석

<1. 퍼널 분석 쿼리>

screen_view(welcome) screen_view(home) screen_view(food_category)

screen_view(restaurant)

screen_view(cart)

click_payment(cart)

```
-- 퍼널 데이터
-- 우리가 사용할 이벤트 => 단계
-- screen_view : welcome, home, food_category, restaurant, cart
-- click_payment
-- step_number : 추후에 정렬을 위해 만들 것
-- 사용할 데이터 : 앱 로그 데이터, GA/Firebase => UNNEST => PIVOT
-- 기간 : 2022-08-01 ~ 2022-08-18
```

```
WITH base AS (
    SELECT
        event_date,
        event_timestamp,
        event_name,
        user_id,
        user_pseudo_id,
        platform,
        MAX(IF(event_param.key = 'firebase_screen', event_param.value.string_value, NULL)) AS firebase_screen,
        #MAX(IF(event_param.key = 'food_id', event_param.value.int_value, NULL)) AS food_id,
        MAX(IF(event_param.key = 'session_id', event_param.value.string_value, NULL)) AS session_id,
    FROM advanced.app_logs
    CROSS JOIN UNNEST(event_params) AS event_param
    WHERE
        event_date BETWEEN '2022-08-01' AND '2022-08-18'
    GROUP BY ALL
), filter_event_and_concat_event_and_screen AS (
    -- event_name + screen (필요한 이벤트만 WHERE 조건에 걸어서 사용)
    SELECT
        * EXCEPT(event_name, firebase_screen, event_timestamp),
        CONCAT(event_name, '-', firebase_screen) AS event_name_with_screen,
        DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul') AS event_datetime
    FROM base
    WHERE
        event_name IN ('screen_view', 'click_payment')
), before_pivot AS (
```

집계한 데이터를 PIVOT. PIVOT 한 형태에서 퍼널 별 전환율을 쉽게 구할 수 있음

```
SELECT
    event_date,
    MAX(IF(event_name_with_screen = 'screen_view-welcome', cnt, NULL)) AS `screen_view-welcome`, #1
    MAX(IF(event_name_with_screen = 'screen_view-home', cnt, NULL)) AS `screen_view-home`, #2
    MAX(IF(event_name_with_screen = 'screen_view-food_category', cnt, NULL)) AS `screen_view-food_category`, #3
    MAX(IF(event_name_with_screen = 'screen_view-restaurant', cnt, NULL)) AS `screen_view-restaurant`, #4
    MAX(IF(event_name_with_screen = 'screen_view-cart', cnt, NULL)) AS `screen_view-cart`, #5
    MAX(IF(event_name_with_screen = 'click_payment-cart', cnt, NULL)) AS `click_payment-cart`, #6
    CAST(MAX(IF(event_name_with_screen = 'screen_view-home', cnt, NULL)) AS FLOAT64) / CAST(MAX(IF(event_name_with_screen = 'screen_view-welcome', cnt, NULL)) AS FLOAT64) AS CVR1, # 2 / 1
    CAST(MAX(IF(event_name_with_screen = 'screen_view-food_category', cnt, NULL)) AS FLOAT64) / CAST(MAX(IF(event_name_with_screen = 'screen_view-home', cnt, NULL)) AS FLOAT64) AS CVR2, # 3 / 2
    CAST(MAX(IF(event_name_with_screen = 'screen_view-restaurant', cnt, NULL)) AS FLOAT64) / CAST(MAX(IF(event_name_with_screen = 'screen_view-food_category', cnt, NULL)) AS FLOAT64) AS CVR3, # 4 / 3
    CAST(MAX(IF(event_name_with_screen = 'screen_view-cart', cnt, NULL)) AS FLOAT64) / CAST(MAX(IF(event_name_with_screen = 'screen_view-restaurant', cnt, NULL)) AS FLOAT64) AS CVR4, # 5 / 4
    CAST(MAX(IF(event_name_with_screen = 'click_payment-cart', cnt, NULL)) AS FLOAT64) / CAST(MAX(IF(event_name_with_screen = 'screen_view-cart', cnt, NULL)) AS FLOAT64) AS CVR5 # 6 / 5
FROM before_pivot
GROUP BY ALL
ORDER BY
    event_date
LIMIT 10
```

```
-- step_number + COUNT
-- step_number : CASE WHEN을 사용해 숫자 지정
# 일자 상관없이 퍼널의 유저 수를 집계한 쿼리 => 일자별로 하기 위해 event_date 추가

SELECT
    event_date, # 일자별로 퍼널별 유저 수 쿼리
    event_name_with_screen,
    CASE
        WHEN event_name_with_screen = 'screen_view-welcome' THEN 1
        WHEN event_name_with_screen = 'screen_view-home' THEN 2
        WHEN event_name_with_screen = 'screen_view-food_category' THEN 3
        WHEN event_name_with_screen = 'screen_view-restaurant' THEN 4
        WHEN event_name_with_screen = 'screen_view-cart' THEN 5
        WHEN event_name_with_screen = 'click_payment-cart' THEN 6
    ELSE NULL
    END AS step_number,
    COUNT(DISTINCT user_pseudo_id) AS cnt
FROM filter_event_and_concat_event_and_screen
GROUP BY ALL
HAVING
    step_number IS NOT NULL
ORDER BY
    event_date
    -- food_detail, search, search_result도 보면서 어떤 흐름으로 고객이 움직이는지 확인해보기
)
-- SELECT
-- *
-- FROM before_pivot
```

<1. 퍼널 분석 시각화>

위 SQL 쿼리를 통해 일자별 퍼널 집계한 결과를 스프레드 시트로 시각화

쿼리 결과

작업 정보

결과

자트

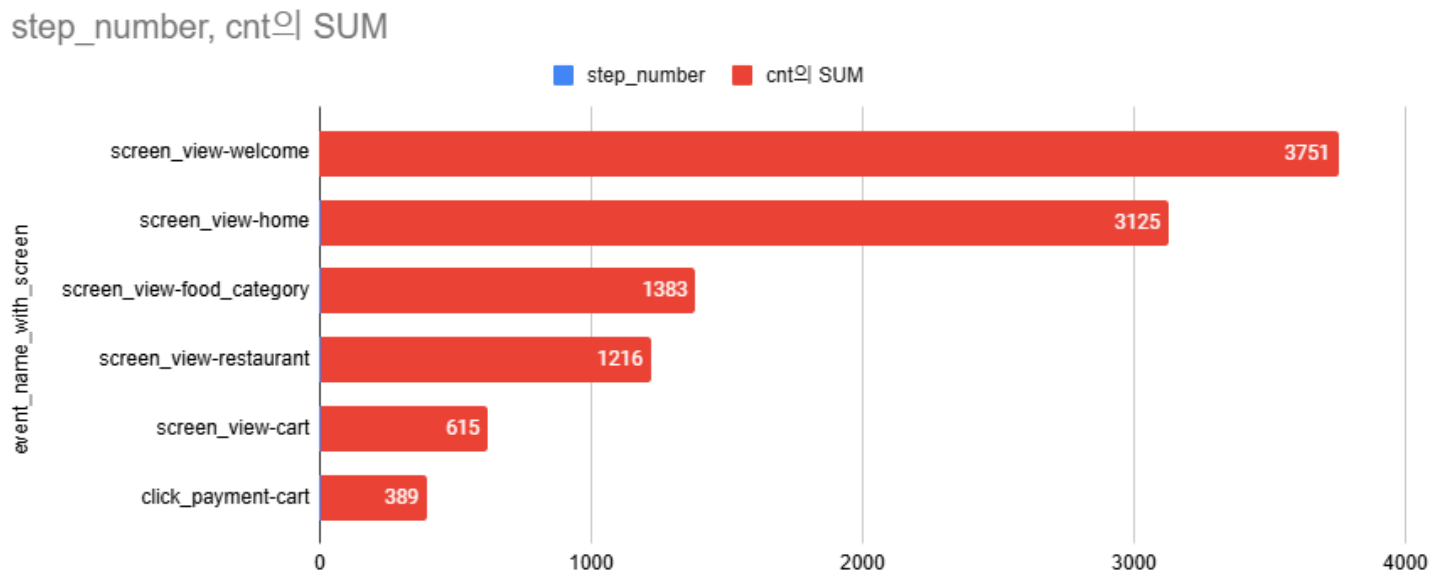
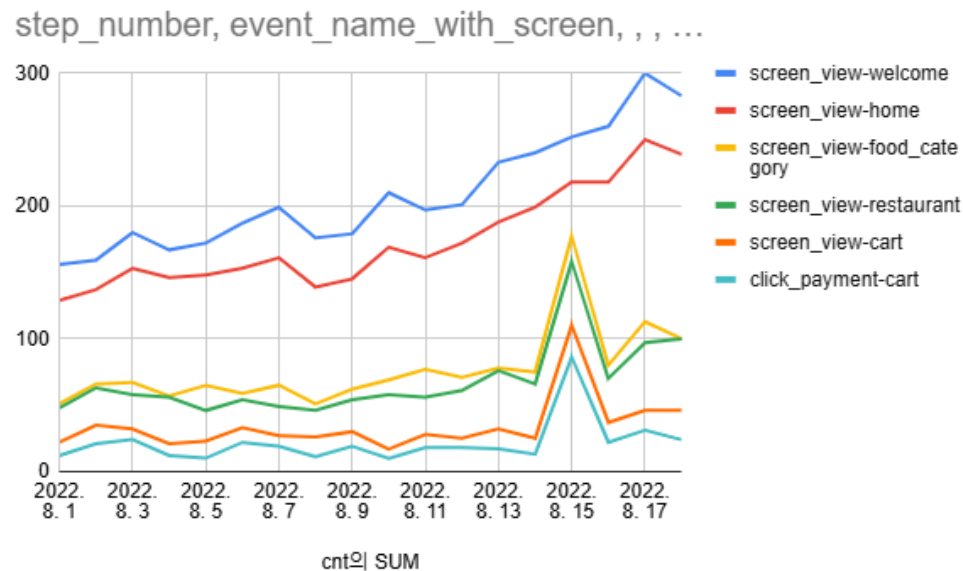
JSON

실행 세부정보

실행 그래프

행	event_date	event_name_with_screen	step_number	cnt
1	2022-08-01	screen_view-welcome	1	156
2	2022-08-01	screen_view-home	2	129
3	2022-08-01	screen_view-food_category	3	51
4	2022-08-01	screen_view-restaurant	4	48
5	2022-08-01	screen_view-cart	5	22
6	2022-08-01	click_payment-cart	6	12
7	2022-08-02	screen_view-welcome		
8	2022-08-02	screen_view-home		
9	2022-08-02	screen_view-food_category		
10	2022-08-02	screen_view-restaurant		

A	B	C
step_number	event_name_with_screen	cnt의 SUM
1	screen_view-welcome	3751
2	screen_view-home	3125
3	screen_view-food_category	1383
4	screen_view-restaurant	1216
5	screen_view-cart	615
6	click_payment-cart	389



Firebase 앱 로그 기반 데이터 분석

<1. 퍼널 분석 해석>

퍼널 데이터 해석하기

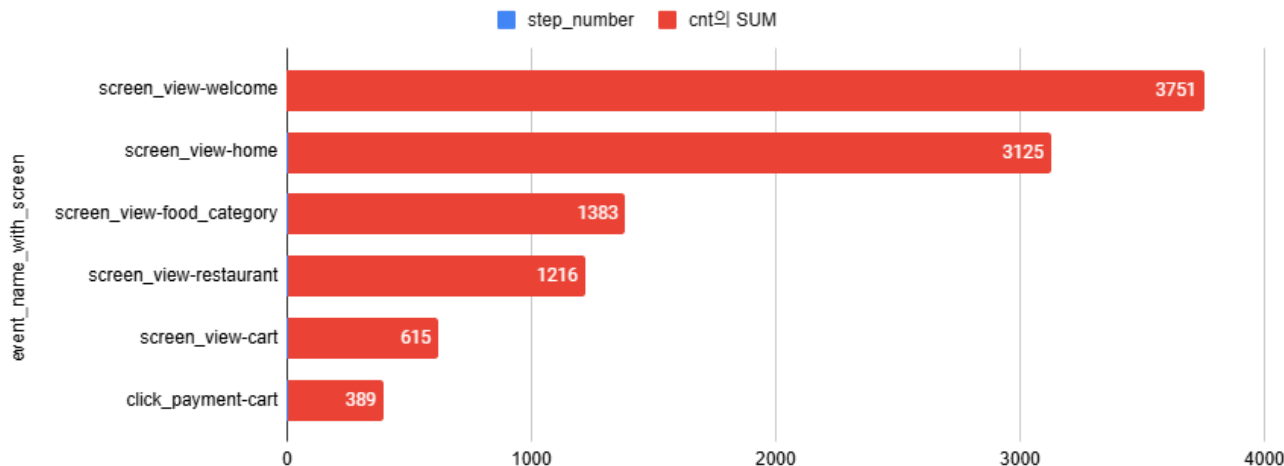
- Home to food_category : 1742명 이탈(직전 대비 55% 이탈)
- Restaurant -> cart : 601명 이탈(직전 대비 49% 이탈)
- cart -> click_payment : 226명 이탈(직전 대비 37% 이탈)

퍼널 관련 여러 아이디어

- 퍼널 단계를 줄이기(Gap 줄이기)
- 퍼널 순서를 바꾸기 : 회원 가입 전에 체험하고, 체험의 어느 단계에서 회원 가입 또는 로그인하도록 설계
- 결제 직전에 추가 상품을 추천해서 업셀링
- 장바구니에 담기 전에 할인 프로모션 안내, 할인 가격 적용해서 노출

쿼리 결과				
작업 정보	결과	차트	JSON	실행 세부정보
실행 그래프				
행	event_date	event_name_with_screen	step_number	cnt
1	2022-08-01	screen_view-welcome	1	156
2	2022-08-01	screen_view-home	2	129
3	2022-08-01	screen_view-food_category	3	51
4	2022-08-01	screen_view-restaurant	4	48
5	2022-08-01	screen_view-cart	5	22
6	2022-08-01	click_payment-cart	6	12
7	2022-08-02	screen_view-welcome	1	159
8	2022-08-02	screen_view-home	2	137
9	2022-08-02	screen_view-food_category	3	66
10	2022-08-02	screen_view-restaurant	4	63

step_number, cnt의 SUM



Firestore 앱 로그 기반 데이터 분석

<2. 리텐션 분석>

SQL 쿼리를 통해 주 별 리텐션 그래프 도출

N day retention

쿼리 결과					
작업 정보		결과	차트	JSON	실행 세부정보
행	diff_of_day	user_cnt			
1	0	35803			
2	1	397			
3	2	366			
4	3	383			
5	4	375			
6	5	381			
7	6	342			
8	7	381			
9	8	398			
10	9	357			

쿼리가 업데이트됨

쿼리 결과					
작업 정보		결과	차트	JSON	실행 세부정보
행	diff_of_day	user_cnt	first_day_user_cnt	retention_rate	
1	0	35803	35803	1.0	
2	1	397	35803	0.011	
3	2	366	35803	0.01	
4	3	383	35803	0.011	
5	4	375	35803	0.01	
6	5	381	35803	0.011	
7	6	342	35803	0.01	
8	7	381	35803	0.011	
9	8	398	35803	0.011	
10	9	357	35803	0.01	

쿼리가 업데이트됨

N day retention : 2022-08-01 ~ 2022-11-03

Weekly retention : 2022-08-01 ~ 2023-08-03

Monthly retention : 2022-08-01 ~ 2022-11-03

Weekly retention

쿼리 결과				
작업 정보		결과	차트	JSON
행	diff_of_week	user_cnt	retention_rate	
1	0	35803	1.0	
2	1	2437	0.068	
3	2	2378	0.066	
4	3	2199	0.061	
5	4	1868	0.052	
6	5	1683	0.047	
7	6	1483	0.041	
8	7	1267	0.035	
9	8	992	0.028	
10	9	838	0.023	

Monthly retention

쿼리 결과			
작업 정보		결과	차트
행	diff_of_month	user_cnt	
1	0	35803	
2	1	6762	
3	2	3335	
4	3	378	

Firestore 앱 로그 기반 데이터 분석

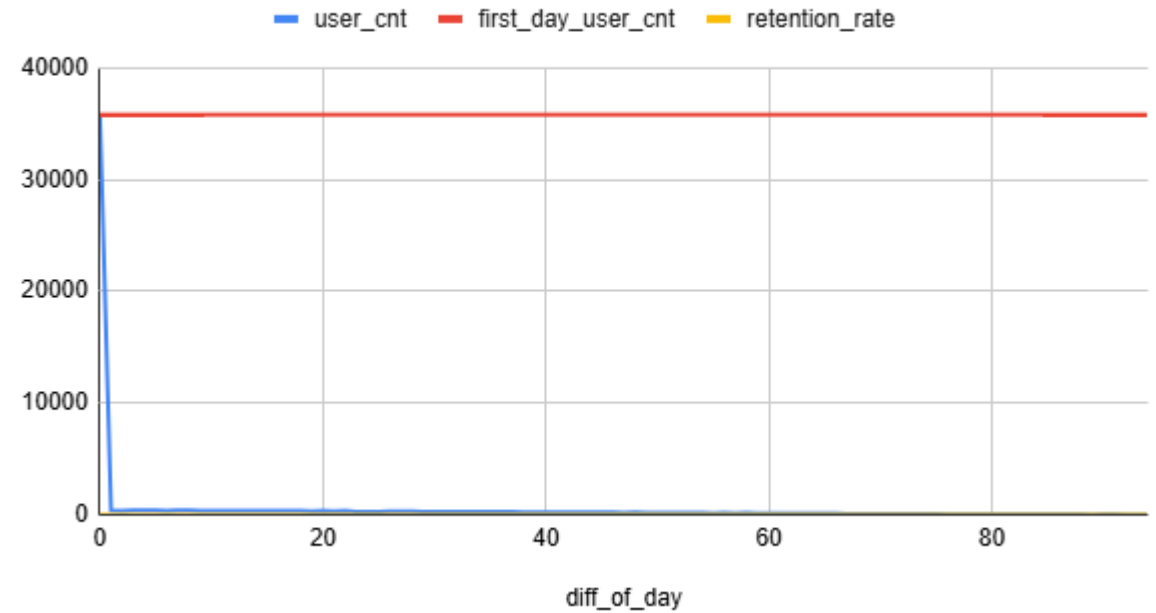
<2. 리텐션 분석>

N day 리텐션을 구해보니, 값이 너무 낮아 보기 어려움

=> Weekly 리텐션(Range)로 수정

N day retention

user_cnt, first_day_user_cnt, retention_rate



Firebase 앱 로그 기반 데이터 분석

<2. 리텐션 분석 쿼리>

```
-- Weekly 리텐션
-- 핵심 : event_date => event_week으로 변경하면 됨
-- 2024-06-30(일) => 2024년 26주차. 2024-06-24(월)도 같은 26주차
-- 데이터를 전처리(가공)하기 위해서 WEEK 함수를 사용할 수도 있고, DATE_TRUNC(일자, 자를 기준)
-- DATE_TRUNC : 2024-06-30 => 2024-06-24
-- WEEK : 26. 2024년 26주차. 주차 별 date를 직관적으로 알기 어렵다
-- WEEK vs ISO_WEEK
-- 주 번호를 계산할 때 사용할 수 있는 함수
-- WEEK : 일요일이 주의 첫 날로 간주. 1월1일이 속한 주가 1주차
-- ISO_WEEK : 월요일이 주의 첫 날로 간주. ISO 8601 국제 표준에 따라 정의. 목요일이 속한 주를 기준으로 주 번호를 지정(목요일이 속하면 그 주의 4일 이상이 포함되기 때문, 연도의 첫 목요일이 있는 주부터 1주차)
-- 2022년 1월 1일(토요일) WEEK : 2022년 1주차. ISO_WEEK 2021년 52주차
-- 첫 목요일은 2022-01-06. 2022-01-03 ~ 2022-01-09가 2022년 1주차
```

```
WITH base AS (
  SELECT
    DISTINCT
      user_id,
      event_name,
      DATE(DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul')) AS event_date,
      DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul') AS event_datetime,
      user_pseudo_id
  FROM advanced.app_logs
  WHERE
    event_date BETWEEN '2022-08-01' AND '2023-08-03'
), first_week_and_diff AS (
  SELECT
    *,
    # DATE_DIFF(event_date, first_date, DAY) AS diff_of_day
    DATE_DIFF(event_week, first_week, WEEK) AS diff_of_week
  FROM (
    SELECT
      DISTINCT
        -- DATE_TRUNC
        user_pseudo_id,
        DATE_TRUNC(MIN(event_date) OVER(PARTITION BY user_pseudo_id), WEEK(MONDAY)) AS first_week,
        DATE_TRUNC(event_date, WEEK(MONDAY)) AS event_week,
        -- event_date
      FROM base
    )
  ), user_counts AS (
  SELECT
    diff_of_week,
    COUNT(DISTINCT user_pseudo_id) AS user_cnt
  FROM first_week_and_diff
  GROUP BY
    diff_of_week
)

SELECT
  diff_of_week,
  user_cnt,
  ROUND(SAFE_DIVIDE(user_cnt, first_cnt), 3) AS retention_rate
FROM (
  SELECT
    *,
    FIRST_VALUE(user_cnt) OVER(ORDER BY diff_of_week) AS first_cnt
  FROM user_counts
)
ORDER BY
  diff_of_week
```

Firestore 앱 로그 기반 데이터 분석

<2. 리텐션 분석 시각화 & 해석>

Retention Curve 해석하기

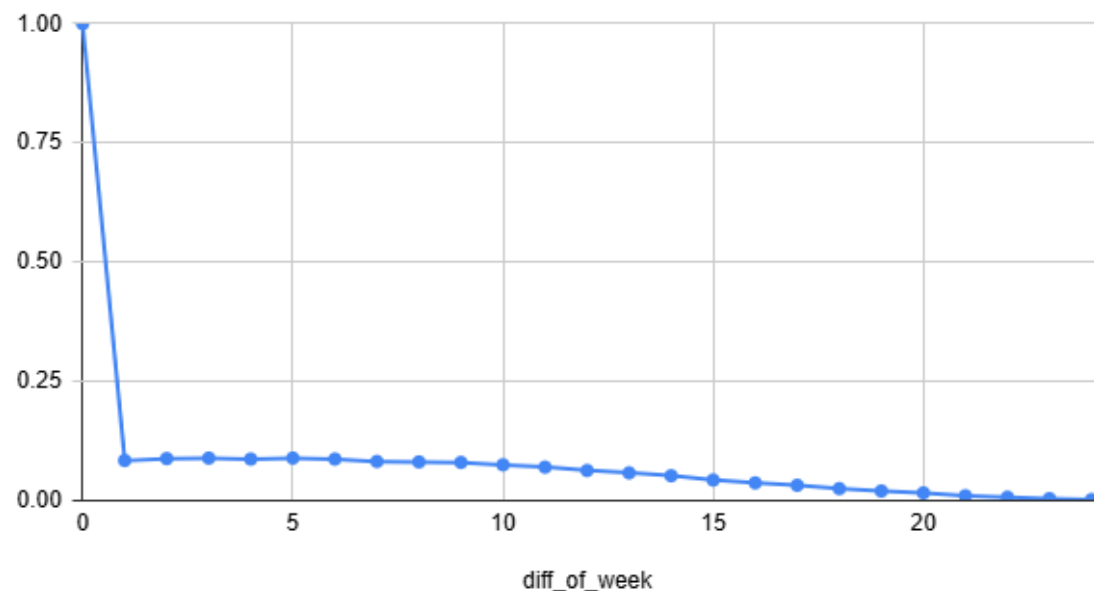
- 1주차에 떨어진다
- 2주차부터 평평한 편이다
- 1주차에 떨어진다는 것은 제품의 핵심 가치(PMF)가 자리잡지 않았을 수 있음
- 그러나 그 후엔 평평함 -> 어떤 유저들은 이 핵심 가치를 경험했을 수 있음

Retention Curve 관련 아이디어

- PMF를 사람들이 더 많이 경험할 수 있도록 핵심 가치 경험 부분을 재점검
- 평평한(Retention Plateau) 부분에 있는 사람들은 주로 어떤 것을 하고 있는지 파악해보기 (정량 분석 + 사용자 인터뷰 등)
- 주기적으로 사용하는 사람들의 특징을 찾고 이 부분을 핵심 가치 경험에 녹여보는 것도 방법
- 1주 뒤 9%만 남는 상황에 마케팅을 하는 것이 좋을까? x

Weekly retention

user_cnt, retention_rate



Firebase 앱 로그 기반 데이터 분석

<3. 코호트 리텐션>

SQL 쿼리를 통해 코호트 리텐션 시각화 추출

데이터 : 2022-08-01 ~ 2022-12-31

쿼리 결과						
작업 정보		결과	차트	JSON	실행 세부정보	실행 그래프
행	first_week ▼	weeks_after_first_we	active_users ▼	cohort_users ▼	retention_rate ▼	
1	2022-08-01	0	1211	1211	1.0	
2	2022-08-01	1	32	1211	0.03	
3	2022-08-01	2	37	1211	0.03	
4	2022-08-01	3	59	1211	0.05	
5	2022-08-01	4	53	1211	0.04	
6	2022-08-01	5	52	1211	0.04	
7	2022-08-01	6	80	1211	0.07	
8	2022-08-01	7	82	1211	0.07	
9	2022-08-01	8	67	1211	0.06	
10	2022-08-01	9	121	1211	0.1	
11	2022-08-01	10	100	1211	0.11	

Firebase 앱 로그 기반 데이터 분석

<3. 코호트 리텐션 쿼리>

```
-- 코호트 리텐션
-- first_week | weeks_after_first_week | active_users | cohort_users | retention_rate

WITH base AS (
  SELECT
    DISTINCT
      user_id,
      event_name,
      DATE(DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul')) AS event_date,
      DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul') AS event_datetime,
      user_pseudo_id
  FROM advanced.app_logs
  WHERE
    event_date BETWEEN '2022-08-01' AND '2022-12-31'
), first_week_and_diff AS (
  SELECT
    *,
    DATE_DIFF(event_week, first_week, WEEK) AS weeks_after_first_week
  FROM (
    SELECT
      DISTINCT
        user_pseudo_id,
        DATE_TRUNC(MIN(event_date) OVER(PARTITION BY user_pseudo_id), WEEK(MONDAY)) AS first_week,
        DATE_TRUNC(event_date, WEEK(MONDAY)) AS event_week,
      FROM base
    )
  )
), user_counts AS (
  SELECT
    first_week,
    weeks_after_first_week,
    COUNT(DISTINCT user_pseudo_id) AS active_users
  FROM first_week_and_diff
  GROUP BY ALL
)

SELECT
  *,
  ROUND(SAFE_DIVIDE(active_users, cohort_users), 2) AS retention_rate
FROM (
  SELECT
    first_week,
    weeks_after_first_week,
    active_users,
    FIRST_VALUE(active_users) OVER(PARTITION BY first_week ORDER BY weeks_after_first_week) AS cohort_users
  FROM user_counts
)
ORDER BY
  first_week, weeks_after_first_week
```

<3. 코호트 리텐션 시각화>

비율 지표 특성상 분자와 분모의 수가 얼마나 되는지도 참고 가능

비율과 숫자를 모두 보며 파악하기 => 너무 작은 데이터인 경우 비율이 높게 나올 수 있음 주의

active_users		weeks_after_first_week																					
first_week		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
2022. 8. 1	1211	32	37	59	53	52	80	82	67	121	138	120	127	134	142	116	129	125	113	130	124	93	
2022. 8. 8	1385	45	59	59	69	93	85	91	107	153	135	128	145	139	130	158	135	154	157	134	117		
2022. 8. 15	1963	66	88	87	131	93	124	164	216	199	185	245	179	205	214	187	238	236	206	169			
2022. 8. 22	2062	97	91	112	107	145	190	231	219	213	227	250	217	216	203	214	259	199	174				
2022. 8. 29	2390	119	145	133	164	211	279	234	227	296	251	240	228	209	245	284	250	217					
2022. 9. 5	2522	161	147	189	210	282	264	267	269	247	258	253	236	260	301	221	177						
2022. 9. 12	3058	174	223	310	356	320	285	363	317	306	279	304	305	367	297	265							
2022. 9. 19	2767	177	225	301	255	299	305	265	282	284	273	303	326	293	226								
2022. 9. 26	2842	236	353	298	285	330	296	284	298	291	294	385	281	215									
2022. 10. 3	3773	408	403	389	443	387	404	367	348	421	464	372	354										
2022. 10. 10	4048	424	431	486	415	428	390	363	427	472	435	307											
2022. 10. 17	3149	316	370	347	293	303	319	298	376	296	254												
2022. 10. 24	2985	329	321	311	283	293	348	362	303	240													
2022. 10. 31	2986	311	340	280	298	319	370	299	234														
2022. 11. 7	2368	246	232	236	253	304	235	191															
2022. 11. 14	2148	209	217	211	260	204	162																
2022. 11. 21	1780	181	203	206	197	147																	
2022. 11. 28	1598	181	189	185	126																		
2022. 12. 5	1641	174	164	155																			
2022. 12. 12	1655	175	131																				
2022. 12. 19	1205	95																					
2022. 12. 26	879																						

Firestore 앱 로그 기반 데이터 분석

<3. 코호트 리텐션 해석>

retention_rate%		weeks_after_first_week																					
first_week		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
2022. 8. 1	1		0.03	0.03	0.05	0.04	0.04	0.07	0.07	0.06	0.1	0.11	0.1	0.1	0.11	0.12	0.1	0.11	0.1	0.09	0.11	0.1	0.08
2022. 8. 8	1		0.03	0.04	0.04	0.05	0.07	0.06	0.07	0.08	0.11	0.1	0.09	0.1	0.1	0.09	0.11	0.1	0.11	0.11	0.1	0.08	
2022. 8. 15	1		0.03	0.04	0.04	0.07	0.05	0.06	0.08	0.11	0.1	0.09	0.12	0.09	0.1	0.11	0.1	0.12	0.12		0.09		
2022. 8. 22	1		0.05	0.04	0.05	0.05	0.07	0.09	0.11	0.11	0.1	0.11	0.12	0.11	0.1	0.1	0.1	0.13	0.1	0.08			
2022. 8. 29	1		0.05	0.06	0.06	0.07	0.09	0.12	0.1	0.09	0.12	0.11	0.1	0.1	0.09	0.1	0.12	0.1	0.09				
2022. 9. 5	1		0.06	0.06	0.07	0.08	0.11	0.1	0.11	0.11	0.1	0.1	0.1	0.09	0.1	0.12	0.09	0.07					
2022. 9. 12	1		0.06	0.07	0.1	0.12	0.1	0.09	0.12	0.1	0.1	0.09	0.1	0.1	0.12	0.1	0.09						
2022. 9. 19	1		0.06	0.08	0.11	0.09	0.11	0.11	0.1	0.1	0.1	0.1	0.1	0.11	0.12	0.11	0.08						
2022. 9. 26	1		0.08	0.12	0.1	0.1	0.12	0.1	0.1	0.1	0.1	0.1	0.14	0.1	0.08								
2022. 10. 3	1		0.11	0.11	0.1	0.12	0.1	0.11	0.1	0.09	0.11	0.12	0.1	0.09									
2022. 10. 10	1		0.1	0.11	0.12	0.1	0.11	0.1	0.09	0.11	0.12	0.11	0.08										
2022. 10. 17	1		0.1	0.12	0.11	0.09	0.1	0.1	0.09	0.12	0.09	0.08											
2022. 10. 24	1		0.11	0.11	0.1	0.09	0.1	0.12	0.12	0.1	0.08												
2022. 10. 31	1		0.1	0.11	0.09	0.1	0.11	0.12	0.1	0.08													
2022. 11. 7	1		0.1	0.1	0.1	0.11	0.13	0.1	0.08														
2022. 11. 14	1		0.1	0.1	0.1	0.12	0.09	0.08															
2022. 11. 21	1		0.1	0.11	0.12	0.11	0.08																
2022. 11. 28	1		0.11	0.12	0.12	0.08																	
2022. 12. 5	1		0.11	0.1	0.09																		
2022. 12. 12	1		0.11	0.08																			
2022. 12. 19	1		0.08																				
2022. 12. 26	1																						

코호트 리텐션 해석하기

- 갑자기 올라간 곳이 있는가? 왜 그럴까? 어떤 기능이 배포되었는가?
- 올라간 특징을 기능으로 만들 수 있는가?
- 초반에 사용한 사람들의 리텐션은 낮음
- 2022-10-03부터 올라가고, 2022-11-28은 모두 준수함
- 그 사이에는 0.1정도의 수치
- 2022-12-19에 감소되고 있어서, 그 다음 주차가 어떻게 될 지 보는 것도 필요

Firebase 앱 로그 기반 데이터 분석

<4. Retain유저 분류 분석>

유저를

신규유저(New) : 제품을 처음 사용하는 유저

기존유저(Current) : 제품을 지속적으로 사용하는 유저

복귀유저(Resurrected) : 과거에 사용 -> 비활성 -> 다시 제품을 사용한 유저

휴면유저(Dormant) : 일정 기간 제품을 사용하지 않은 비활성화 사용자

4가지 분류로 type을 설정해 보았습니다.

데이터 : 2022-08-01 ~ 2023-08-03

쿼리 결과

작업 정보		결과	차트	JSON	실행 세부정보	실행 그래프	
행	event_week	weeks_after_first_use	user_type	user_count	cohort_users	retention_rate	
1	2022-08-01	0	New	1211	1211	1.0	
2	2022-08-08	1	New	1385	1385	1.0	
3	2022-08-08	1	Current	32	1385	0.023	
4	2022-08-08	1	Dormant	25	1385	0.018	
5	2022-08-15	2	New	1963	1963	1.0	
6	2022-08-15	2	Current	45	1963	0.023	
7	2022-08-15	2	Resurrected	37	1963	0.019	
8	2022-08-15	2	Dormant	75	1963	0.038	
9	2022-08-22	3	New	2062	2062	1.0	
10	2022-08-22	3	Current	68	2062	0.033	
11	2022-08-22	3	Resurrected	116	2062	0.056	
12	2022-08-22	3	Dormant	151	2062	0.073	
13	2022-08-29	4	New	2390	2390	1.0	
14	2022-08-29	4	Current	103	2390	0.043	
15	2022-08-29	4	Resurrected	194	2390	0.081	
16	2022-08-29	4	Dormant	245	2390	0.103	
17	2022-09-05	5	New	2522	2522	1.0	
18	2022-09-05	5	Current	128	2522	0.051	
19	2022-09-05	5	Resurrected	290	2522	0.115	
20	2022-09-05	5	Dormant	367	2522	0.146	
21	2022-09-12	6	New	3058	3058	1.0	
22	2022-09-12	6	Current	187	3058	0.061	

Firebase 앱 로그 기반 데이터 분석

<4. Retain유저 분류 쿼리>

```
SELECT
  event_week,
  DATE_DIFF(event_week, first_week, WEEK) AS weeks_after_first_week,
  user_type,
  user_count,
  cohort_users,
  ROUND(SAFE_DIVIDE(user_count, cohort_users), 3) AS retention_rate
FROM (
  SELECT
    DATE(event_week) AS event_week,
    DATE(MIN(event_week) OVER()) AS first_week,
    user_type,
    user_count,
    MAX(CASE WHEN user_type = 'New' THEN user_count END) OVER(PARTITION BY DATE(event_week)) AS cohort_users
  FROM user_counts
  ORDER BY
    event_week,
    CASE user_type
      WHEN 'New' THEN 1
      WHEN 'Current' THEN 2
      WHEN 'Resurrected' THEN 3
      WHEN 'Dormant' THEN 4
    END
)
```

```
WITH base AS (
  SELECT
    DISTINCT
      user_id,
      DATE(DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul')) AS event_date,
      DATE_TRUNC(DATETIME(TIMESTAMP_MICROS(event_timestamp), 'Asia/Seoul'), WEEK(MONDAY)) AS event_week,
      user_pseudo_id
  FROM advanced.app_logs
  WHERE
    event_date BETWEEN '2022-08-01' AND '2023-08-03'
), user_activity AS (
  SELECT
    user_pseudo_id,
    event_week,
    LAG(event_week, 1) OVER(PARTITION BY user_pseudo_id ORDER BY event_week) AS prev_week,
    MIN(event_week) OVER(PARTITION BY user_pseudo_id) AS first_week
  FROM base
), user_classification AS (
  SELECT
    user_pseudo_id,
    event_week,
    CASE
      WHEN event_week = first_week THEN 'New'
      WHEN prev_week IS NOT NULL AND DATE_DIFF(event_week, prev_week, WEEK) = 1 THEN 'Current'
      WHEN prev_week IS NOT NULL AND DATE_DIFF(event_week, prev_week, WEEK) > 1 THEN 'Resurrected'
      ELSE 'Dormant'
    END AS user_type
  FROM user_activity
), user_counts AS (
  SELECT
    event_week,
    user_type,
    COUNT(DISTINCT user_pseudo_id) AS user_count
  FROM user_classification
  GROUP BY ALL
  ORDER BY
    event_week,
    user_type
)
```

Firebase 앱 로그 기반 데이터 분석

<4. Retain유저 분류 코호트 시각화 & 해석>

retention_rate% weeks_after_first_week																									
user_type	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Resurrected			0.019	0.056	0.081	0.115	0.175	0.217	0.331	0.382	0.529	0.686	0.846	1.087	1.329	1.584	1.97	2.265	2.539	2.938	3.4	3.988	4.599	5.338	6.125
New	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Dormant		0.018	0.038	0.073	0.103	0.146	0.197	0.244	0.347	0.395	0.564	0.772	0.891	1.106	1.354	1.574	1.935	2.205	2.454	2.86	3.439	3.823	4.363	4.983	5.768
Current		0.023	0.023	0.033	0.043	0.051	0.061	0.08	0.084	0.088	0.142	0.226	0.21	0.233	0.304	0.306	0.347	0.375	0.378	0.459	0.641	0.565	0.604	0.651	0.766

2022-08-01 ~ 2023-08-03 기간으로 설정하여, 개인적인 의견으로 Retain User를 쪼개 보았습니다.

배달어플은 유저들의 사용 간격이 짧다고 생각하기 때문에, 주의 주기로 리텐션으로 설정했습니다. 저는 바로 전 주에 사용했고, 그 다음주에 사용했다면 기존유저(Current)로 설정했습니다.

2주 이상 텀을 두고 다시 사용했다면 복귀유저(Resurrected)로 설정했습니다.

처음 들어가고 사용하지 않았다면 휴면유저(Dormant)로 설정했습니다.

주차마다 '신규/기존/복귀/휴면' 유저를 새로 계산하는 쿼리입니다.

Firestore 앱 로그 기반 데이터 분석

<4. Retain유저 분류 코호트 시각화 & 해석>

retention_rate% weeks_after_first_week																									
user_type	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Resurrected			0.019	0.056	0.081	0.115	0.175	0.217	0.331	0.382	0.529	0.686	0.846	1.087	1.329	1.584	1.97	2.265	2.539	2.938	3.4	3.988	4.599	5.338	6.125
New	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Dormant		0.018	0.038	0.073	0.103	0.146	0.197	0.244	0.347	0.395	0.564	0.772	0.891	1.106	1.354	1.574	1.935	2.205	2.454	2.86	3.439	3.823	4.363	4.983	5.768
Current		0.023	0.023	0.033	0.043	0.051	0.061	0.08	0.084	0.088	0.142	0.226	0.21	0.233	0.304	0.306	0.347	0.375	0.378	0.459	0.641	0.565	0.604	0.651	0.766

데이터를 살펴보면,

1) 신규유저(New) : 제품을 처음 사용하는 유저

출시(2022-08-01을 출시기간으로 생각하면)로부터 현재(2023-01-16 => 0803으로 설정했지만 0116까지가 끝입니다.)까지의 신규유저 수의 변화를 살펴볼 것입니다.

런칭부터 10주까지의 신규유저의 수는 주춤한 경우도 있지만 평균 4~500명 정도 증가했습니다.

하지만, 11주에 4000명대에서 3100명으로 900명이 급감한 이후로 계속 하락세를 타고 24주차 현재 신규 가입자의 수는 500명 남짓하고 있습니다.

11주차부터의 마케팅 효과는 빛을 보지 못하고 있는 것 같습니다.

코호트 리텐션 시각화를 보자면,

2) 기존유저(Current) : 제품을 지속적으로 사용하는 유저(바로 전 주와 이번 주)

출시로부터 현재까지 줄곧 주차마다의 신규유저 대비 기존유저의 비율이 증가하고 있습니다.

특히, 24주차에 가입한 신규유저 대비 기존유저의 비율은 76%나 차지하고 있습니다.

시간이 흐름에 따라 점차 어플이 안정화되면서 신규유저들이 만족을 느끼는 비율이 늘어가고 있다고 보입니다.

Firebase 앱 로그 기반 데이터 분석

<4. Retain유저 분류 코호트 시각화 & 해석>

retention_rate% weeks_after_first_week																									
user_type	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Resurrected			0.019	0.056	0.081	0.115	0.175	0.217	0.331	0.382	0.529	0.686	0.846	1.087	1.329	1.584	1.97	2.265	2.539	2.938	3.4	3.988	4.599	5.338	6.125
New	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Dormant		0.018	0.038	0.073	0.103	0.146	0.197	0.244	0.347	0.395	0.564	0.772	0.891	1.106	1.354	1.574	1.935	2.205	2.454	2.86	3.439	3.823	4.363	4.983	5.768
Current		0.023	0.023	0.033	0.043	0.051	0.061	0.08	0.084	0.088	0.142	0.226	0.21	0.233	0.304	0.306	0.347	0.375	0.378	0.459	0.641	0.565	0.604	0.651	0.766

3) 복귀유저(Resurrected) : 과거에 사용 -> 비활성화 -> 다시 제품을 사용한 유저(2주 후 복귀)

복귀유저는 12주차를 분기로 13주차부터는 신규가입자보다 복귀유저의 수가 많아지기 시작했습니다.

24주차에는 신규유저 대비 복귀유저는 6배가 많습니다.

복귀유저가 기존유저로 안정되는 현황을 추출할 수 있는 쿼리는 조금 더 연구해 보겠습니다.

4) 휴면유저(Dormant) : 일정기간 제품을 사용하지 않은 비활성화 사용자

신규유저 대비 휴면유저는 꾸준히 늘어가고 있습니다(휴면유저가 누적됨).

24주차 신규가입자의 5.7배가 휴면유저 입니다.